



RUHR-UNIVERSITÄT BOCHUM

DATA REPRESENTATION

Computer Architecture

Agenda

- Number Systems
- Positive and Negative Numbers
- Architecture and Assembly
- Binary Arithmetic
- Logic Operations

Number Systems

Let's look at numbers

A digit is one of the 10 symbols:

0, 1, 2, 3, 4, 5, 6, 7, 8, 9

A number is a sequence of digits

$$d_n d_{n-1} \dots d_1 d_0$$

The value of the number is:

$$\sum_{i=0}^n d_i 10^i$$

3	2	7
Hundreds	Tens	Units

$$3 \times 100 + 2 \times 10 + 7 \times 1$$

$$3 \times 10^2 + 2 \times 10^1 + 7 \times 10^0$$

Let's look at numbers

A digit is one of the **10** symbols:

0, 1, 2, 3, 4, 5, 6, 7, 8, 9

A number is a sequence of digits

$$d_n d_{n-1} \dots d_1 d_0$$

The value of the number is:

$$\sum_{i=0}^n d_i 10^i$$

Decimal numbers

Base 10

3	2	7
Hundreds	Tens	Units

$$3 \times 100 + 2 \times 10 + 7 \times 1$$

$$3 \times 10^2 + 2 \times 10^1 + 7 \times 10^0$$

Positional number systems

A **decimal** digit is one of the **10** symbols:

0, 1, 2, 3, 4, 5, 6, 7, 8, 9

A number is a sequence of digits

$$d_n d_{n-1} \dots d_1 d_0$$

The value of the number is:

$$\sum_{i=0}^n d_i 10^i$$

A **base b** digit is one of the **b** symbols:

0, 1, ..., ($b - 1$)

A number is a sequence of digits

$$d_n d_{n-1} \dots d_1 d_0$$

The value of the number is:

$$\sum_{i=0}^n d_i b^i$$

Positional number systems

How much is $(432)_5$?

$$4 \times 5^2 + 3 \times 5^1 + 2 \times 5^0$$

$$4 \times 25 + 3 \times 5 + 2$$

$$100 + 15 + 2$$

$$117$$

A base b digit is one of the b symbols:
 $0, 1, \dots, (b - 1)$

A number is a sequence of digits

$$d_n d_{n-1} \dots d_1 d_0$$

The value of the number is:

$$\sum_{i=0}^n d_i b^i$$

Binary System

Base: 2

Digits: 0,1

Prefix: 0b

How much is 0b101?

2^2	2^1	2^0
1	0	1

$$\begin{aligned}1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 \\&= 4 + 1 \\&= 5\end{aligned}$$

Digital data

- Computers use electrical currents to represent data
 - Only consider existence, not magnitude.
- Existence can signal one of two values – either exists or does not exist.
- We use Binary Digits (bits) to model the data
 - Current flows → value 1
 - Current does not flow → 0
- Contrast with analog systems that use magnitude
 - Voltage, current, etc.

Grouping data

- One bit is a very small unit. Need to talk about groups of bits.
- Byte – 8 bits
- Word: depends on context
 - Anyone except Intel and Microsoft: Native data size of the processor:
 - Most computers: 64 bits
 - Microcontrollers: 32 bits
 - Intel/Microsoft:
 - Word: 16 bits
 - Dword: 32 bits
 - Qword: 64 bits

Unless stated otherwise,
in this course we use
32-bit words

MSB and LSB



Decimal to Binary: Conversion Table

1. Determine how many bits we need to represent the number

➡ $n: 2^{n-1} \leq \text{number} < 2^n$

2. Compute which bits are set to 1 and which are set to 0

Start with the most significant bit and successively try to subtract all powers of two:

➡ Power is larger than the number: Set this bit to 0

➡ Power is smaller than or equal to the number: Set this bit to 1 and subtract the power of 2 from the number

Example

Convert 85 to Binary

1.

$2^7 = 128$	$2^6 = 64$	$2^5 = 32$	$2^4 = 16$	$2^3 = 8$	$2^2 = 4$	$2^1 = 2$	$2^0 = 1$
-------------	------------	------------	------------	-----------	-----------	-----------	-----------

128 > 85:
We do not
need this bit



We need 7 bits to represent the number

Example

Convert 85 to Binary

2.

$2^7 = 128$	$2^6 = 64$	$2^5 = 32$	$2^4 = 16$	$2^3 = 8$	$2^2 = 4$	$2^1 = 2$	$2^0 = 1$
	$85 \geq 64?$ $85 - 64 = 21$	$21 \geq 32?$	$21 \geq 16?$ $21 - 16 = 5$	$5 \geq 8?$	$5 \geq 4?$ $5 - 4 = 1$	$1 \geq 2?$	$1 \geq 1?$ $1 - 1 = 0$

Binary to Decimal: Conversion Table

1. Put the binary number back in the conversion table
2. Add all powers of 2 where the bit is 1

Example

$2^6 = 64$	$2^5 = 32$	$2^4 = 16$	$2^3 = 8$	$2^2 = 4$	$2^1 = 2$	$2^0 = 1$
1	1	1	0	0	0	1

$$\begin{aligned} &64 + 32 + 16 + 1 \\ &= 113 \end{aligned}$$

Decimal to Binary: Divide and Subtract

1. Divide number by 2

➡ Result is even: write it as d

➡ Result is odd: write it as $(d - 1) + 1$

2. Repeat with $(d - 1)$ or d until $d = 0$

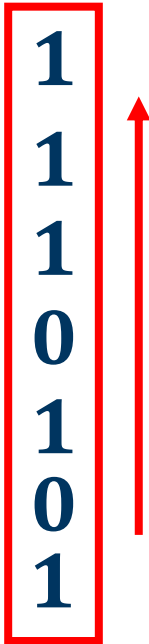
3. Result can be read from the remainders

Decimal to Binary: Divide and Subtract

1. Divide the number by two, record the remainder
2. Repeat until you reach 0.
3. Result is the sequence of remainders

Example: 87 to binary

87	:	2	=	43	1
43	:	2	=	21	1
21	:	2	=	10	1
10	:	2	=	5	0
5	:	2	=	2	1
2	:	2	=	1	0
1	:	2	=	0	1



Binary to Decimal: Multiply and Add

1. Start with 0 and add the MSB to it
2. Multiply the sum by 2 and add the next bit to the product
3. Repeat until all bits have been used
4. The last product is the result

Example: 1110001 to decimal

$$0 + 1 = 1$$

$$1 \times 2 + 1 = 3$$

$$3 \times 2 + 1 = 7$$

$$7 \times 2 + 0 = 14$$

$$14 \times 2 + 0 = 28$$

$$28 \times 2 + 0 = 56$$

$$56 \times 2 + 1 = 113$$

And now it's your turn...

Convert the signed number 11001100 into its decimal system representation using the 'Multiply and Add' method.

Hexadecimal System

Base: 16

Digits: 0-9,A-F

A-F represent the digit values 10-15

Prefix: 0x

How much is 0xA3E?

16^2	16^1	16^0
A	3	E

$$\begin{aligned} & 10 \times 16^2 + 3 \times 16^1 + 14 \times 16^0 \\ &= 2560 + 48 + 14 \\ &= 2622 \end{aligned}$$

Binary to Hexadecimal

Observation: We can represent 0-15 using four bits

Example:

0101 1110 0001 1101



5 *E* 1 *D*

Octal System

Base: 8

Digits: 0-7

Prefix: 0o or just 0

How much is 0756:

8^2	8^1	8^0
7	5	6

$$\begin{aligned}7 \times 8^2 + 5 \times 8^1 + 6 \times 8^0 \\&= 448 + 40 + 6 \\&= 494\end{aligned}$$

Binary to Octal

Observation: We can represent 0-7 using three bits

Example:

111 110 001 101



7 6 1 5

Binary Arithmetic

Binary Addition

1. Column addition
2. Check for overflow and add a bit, if needed

$$\begin{array}{r} 1010 \\ + 1011 \\ \hline 1 \quad 1 \end{array}$$

Binary Multiplication

Long multiplication

$$\begin{array}{r} 1010 \times 101 \\ \hline 1010 \\ 1010 \\ 1010 \\ \hline 110010 \end{array}$$

Binary Division

Long division

$$\begin{array}{r} 1111 \div 11 = 0101 \\ \underline{1} \\ 11 \\ \underline{-11} \\ 001 \\ 11 \\ \underline{-11} \\ 00 \end{array}$$

Signed Numbers

Unsigned Integers

Fixed word size

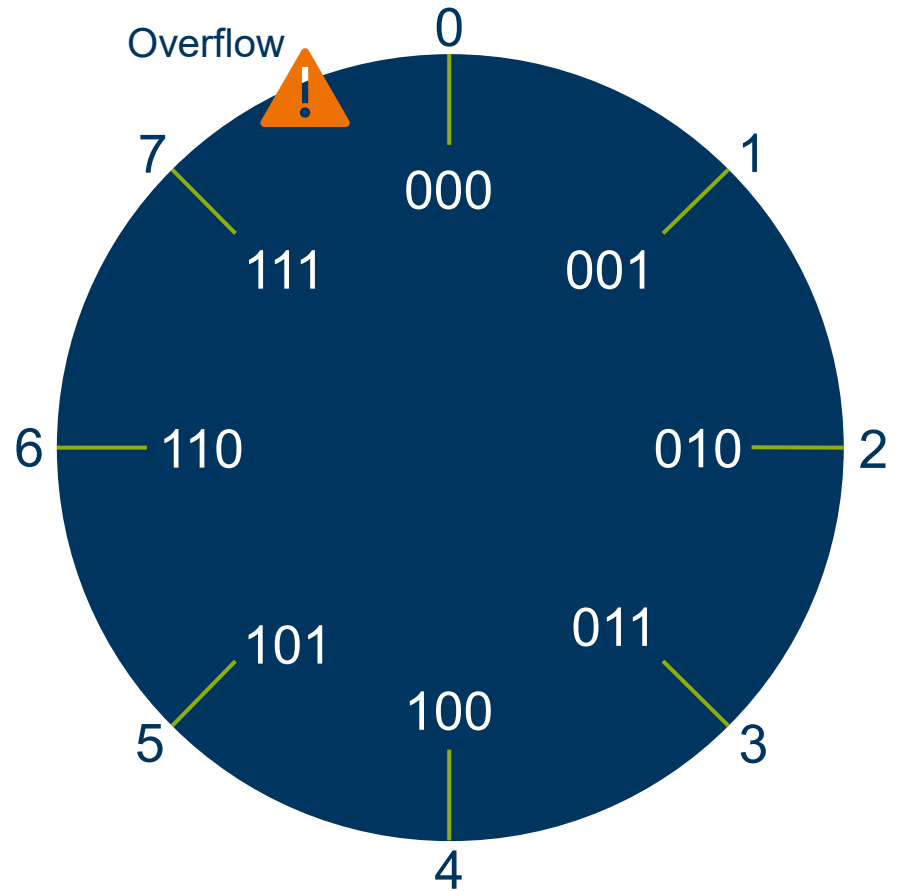
Range: $0 \dots 2^n$ (here $n = 3$)

Formula: $\sum_{i=0}^{n-1} b_i 2^i$

Overflow if the result is above 2^n

Arithmetic modulo 2^n

No negative numbers



Zero Extension

To extend an unsigned number we fill the MSB with zeroes

6 Bit:

101010



12 Bit:

000000101010

Sign and Magnitude

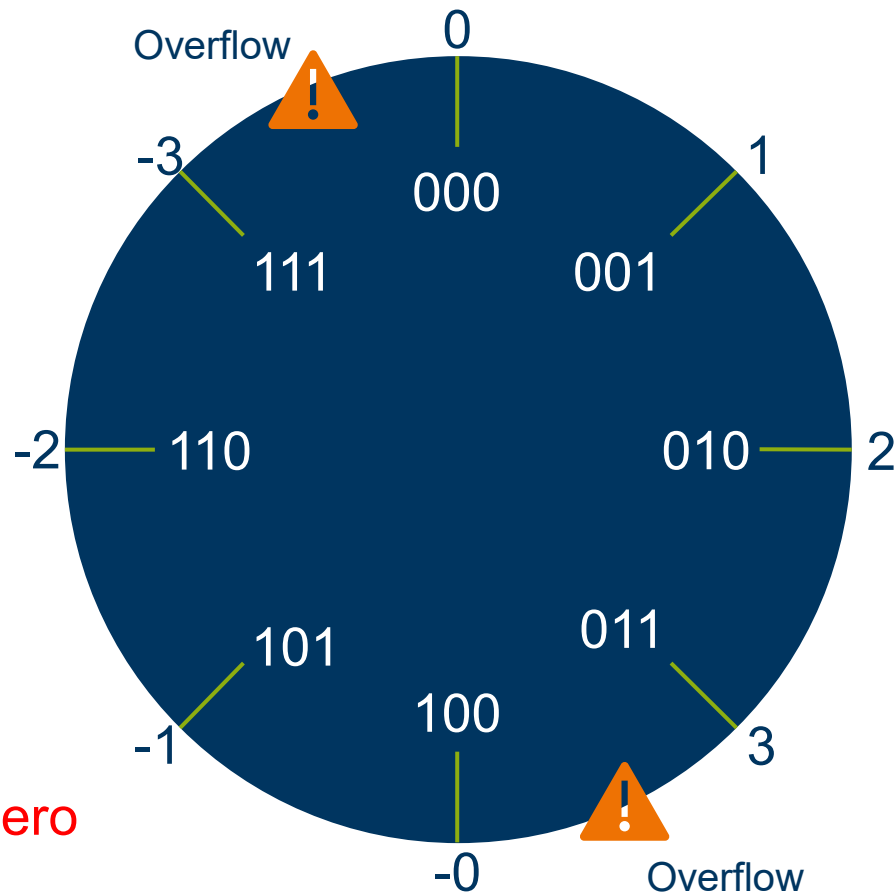
Positive and negative numbers

MSB is used for the Sign

Range: $-(2^{n-1}-1) \dots 2^{n-1}-1$

Formula: $(1 - 2b_{n-1}) \sum_{i=0}^{n-2} b_i 2^i$

Two different representations of zero



Ones' Complement

Positive and negative numbers

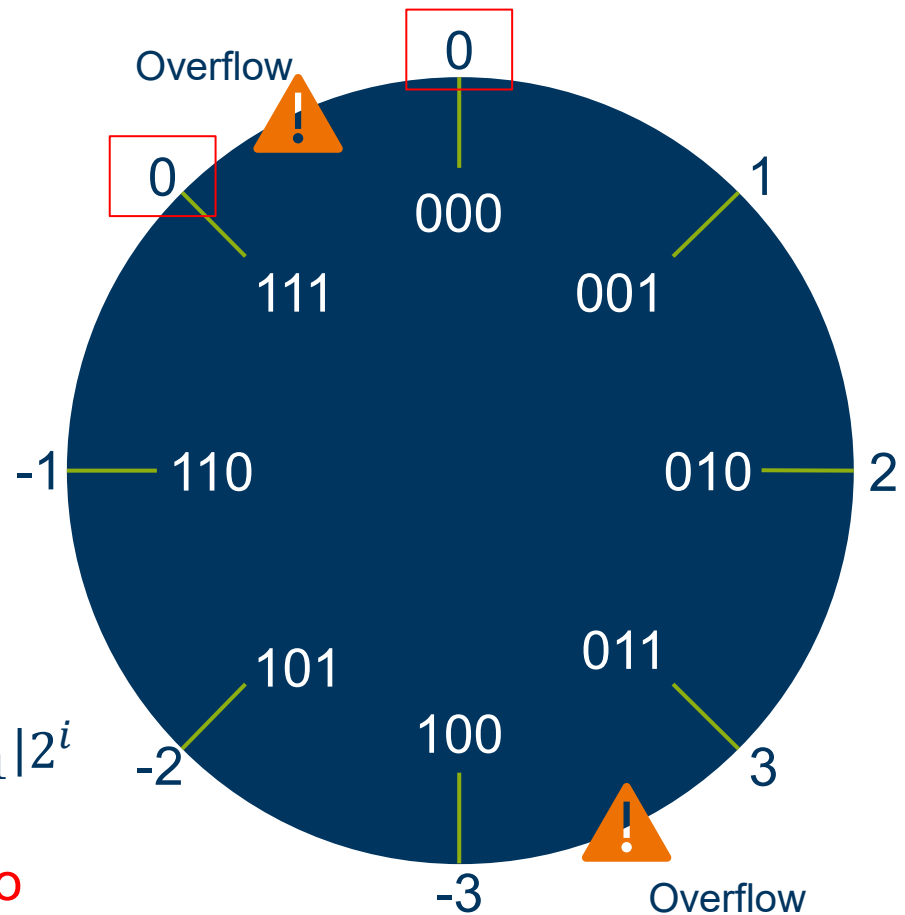
MSB is used for the sign

Invert bits to negate numbers

Range: $-(2^{n-1}-1) \dots 2^{n-1}-1$

Formula: $(1 - 2b_{n-1}) \sum_{i=0}^{n-2} |b_i - b_{n-1}| 2^i$

Two different representations of zero



Two's Complement

Positive and negative numbers

Negative numbers – backwards from 0

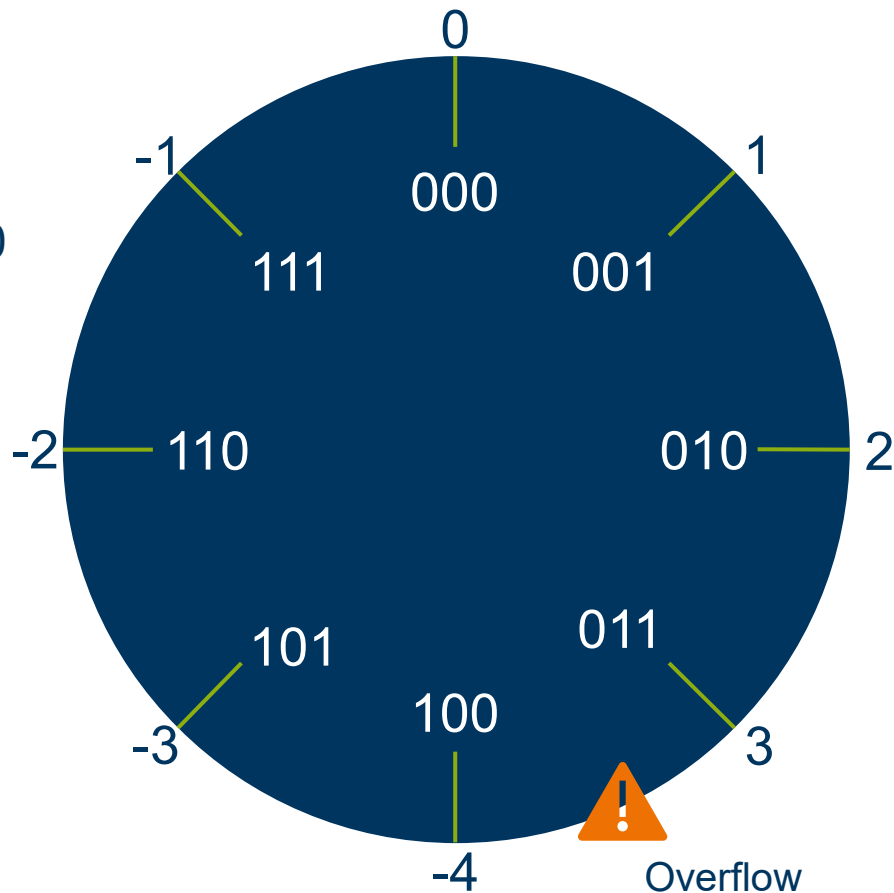
Range: $-(2^{n-1}) \dots 2^{n-1} - 1$

No more signed zero

Formula: $-b_{n-1}2^{n-1} + \sum_{i=0}^{n-2} b_i 2^i$

Arithmetic modulo 2^n still works

Standard representation for integers



Computing the Inverse of a Number

Positive:



Ones' Complement:



Two's Complement:

0110110111



Inverse

1001001000



Add 1

1001001001

Subtraction is just addition with the inverse

And now it's your turn...

What is the binary representation of -29?
Use 6 bit for your answer.

Sign extension

To extend a signed number we fill the MSB with its sign

6 Bit:

101010

001010



12 Bit:

11111101010

000000001010

Logic Operations

AND

Evaluates to 1 only if both operands are set to 1

A	B	$A \wedge B$
0	0	0
0	1	0
1	0	0
1	1	1

OR

Evaluated to 1 if at least one operand is set to 1

A	B	$A \vee B$
0	0	0
0	1	1
1	0	1
1	1	1

XOR

Evaluates to 1 if exactly one operand is set to 1

A	B	$A \vee B$
0	0	0
0	1	1
1	0	1
1	1	0

NOT

Evaluates to the bit's
inverse

Can be computed as XOR
with 1

A	$\neg A$
0	1
1	0



A	B	$A \vee B$
0	0	0
0	1	1
1	0	1
1	1	0

Summary

- How to convert decimal to binary and vice versa
- How to convert binary to octal and hexadecimal
- What unsigned and signed integers are
- How negative integers are represented
- How to add, subtract, multiply and divide binary numbers
- Basic logic gates